Choosing a protocol for IoT device management, telemetry and OTA

MQTT vs LwM2M · conclusions from the embedded-C protocol playground (Mosquitto 2.1.2, Leshan 2.0.0-M18) · October 2026

Recommendation. For **device management and OTA**, LwM2M is the better fit. For **telemetry**, either works; the right choice depends on the backend and how the device is powered and connected. A common production setup is **LwM2M for management and OTA**, plus **LwM2M or MQTT for telemetry**.

DEVICE MANAGEMENT

LwM2M

Standard objects and server-initiated Read / Write / Execute / Observe.

OTA

LwM2M

Standardised Firmware Update object (/5). MQTT has no standard OTA.

TELEMETRY

Either

LwM2M for sleepy cellular devices; MQTT for always-on devices and broker-based backends.

1. What the playground measured

All numbers were measured on the development PC over loopback, with real servers and the playground's C devices. They show **protocol overhead**, not real radio behaviour.

METRIC	MQTT (QOS 1)	LWM2M (CON NOTIFY)	WHAT IT POINTS TO
Wire bytes per telemetry message	159 207 with TLS	37 95 with DTLS	LwM2M is cheaper per message. Part of the gap is that it sent one value while MQTT sent the whole JSON document.
Security handshake	2812 B 1 round trip (TLS 1.3 + certificate)	601 B 3 round trips (DTLS-PSK)	LwM2M uses fewer bytes; MQTT's TLS 1.3 needs fewer round trips.
Time to ready, security on	12.7 ms	25.3 ms	Both are small on loopback; round trips dominate on a real network.
Median latency, clean network	0.38 ms	1.17 ms	Both are negligible.
P95 latency at 10 % packet loss	213 ms	6.2 s	MQTT recovers faster: TCP retransmits after about 200 ms, CoAP waits 2–3 s and backs off.
Delivery at 10 % packet loss	100 %	100 %	Both were reliable.

> The tail-latency gap comes from the default CoAP timers, which can be tuned (the playground's `--ack-timeout` option). If a reading arriving a few seconds late doesn't matter, the gap doesn't matter either.

2. Why LwM2M fits device management and OTA

These points come from how the protocols are designed, not from the measurements above.

- **The management model is standardised.** Device info, battery, reboot, connectivity and firmware are defined objects (`/3`, `/4`, `/5`). With MQTT you design all of that yourself: topics, payloads and commands.
- **The server can act on the device.** Read, Write, Execute and Observe all worked against Leshan in the playground. MQTT needs its own request/response conventions on top of publish/subscribe.
- **OTA is standardised.** Object 5 defines the steps: the server provides the package or its URI, the device downloads it in blocks, reports its state, and the server triggers the update. MQTT has no standard OTA, so you build your own or use a cloud vendor's job system.
- **It is built for sleepy cellular devices.** It runs over UDP with no permanent connection, with registration lifetimes and queue mode for PSM/eDRX. MQTT needs a live TCP connection with keep-alives, which costs power and data and can be dropped by carrier NAT timeouts.

3. When MQTT is the better choice for telemetry

- The device is **always powered and connected**, and the lowest latency on lossy links matters.
- The backend already uses MQTT, such as AWS IoT, Azure IoT or another broker-based pipeline.
- You need **fan-out**: many consumers subscribing to the same data stream.

4. Recommendation for the STM32 + EC200U product

Battery-powered or sleepy device

LwM2M only

Use LwM2M for everything: management, OTA through Object 5, and telemetry through Observe/Notify or Send.

Mains-powered, MQTT cloud backend

LwM2M + MQTT

Use MQTT for telemetry and LwM2M for management and OTA. If only one protocol is allowed, an OTA flow over MQTT has to be built and maintained in-house.

5. Limits of this evidence and next steps

- The playground's OTA is a generic chunked HTTP download, not LwM2M Object 5, so it does not compare OTA between the protocols directly.
- Nothing ran over a real radio. Loopback plus simulated latency and loss shows mechanisms, not field performance, power use or data cost.
- Byte counts depend on payload design (one LwM2M resource vs a full JSON document over MQTT).

Next steps: implement LwM2M Object 5 in the playground, then repeat the benchmark, loss and security runs over the EC200U on the target network to turn this reasoning into measured evidence.

Source: MQTT vs LwM2M Playground — results in `results/runs.jsonl`; design and methodology in `docs/playground.tex`.